



Distributed algorithms

Dr. Benoît PIRANDA

Ph.D. Associate Professor HDR

Member of Computer Science Department of Femto-ST institute

benoit.piranda@femto-st.fr



Outline



- Distributed algorithm principle
 - Basic flooding
 - Flooding with acknowledgment
 - Create a spanning tree
 - Minimal spanning tree
- Examples of distributed algorithms
 - Leader election
 - Elect the centroid
- Implement a simulator of distributed systems

Distributed algorithm principle



- We consider a set of agents
 - Computing capability
 - Local memory
 - Local program
 - Connected to some other agents
 - Can exchange messages with embedded data
 - A message may take time to be sent.
 - Sensors and actuators
 - Can receive events from the neighborhood
 - Can actuate to change the environment

Create a graph where agents are the vertices and connections the links

Exemple of distributed systems



- Set of IOT connected objects
 - Agent = connected object
 - Able to communicate to a subset of agents
 - Local network of connected objects
 - Usually static
- Programmable Matter
 - Agent = robot
 - Placed in a regular lattice
 - Able to communicate with direct neighbors
 - Able to move (change the topology of the graph)

Local knowledge of the graph



- Each agent knows only:
 - Its local data.
 - Information that direct neighbors send to it.
- A message stores:
 - ID: kind of the message, a program may send messages with different meaning.
 - Embedded data
- Action of agents
 - sendMessage
 - receiveMessage in its message callback function
 - Change state (color, blink...)

Need distributed programs



- All agents are running the same program simultaneously
- Distributed program
 - All agents have local information
 - When they receive a message or an event
 - They can update their local data (please use mySomething variable name)
 - They can activate an actuator (change color...)
 - They can send message to some of their direct neighbors
- The problem is to program local behaviors to get a global result

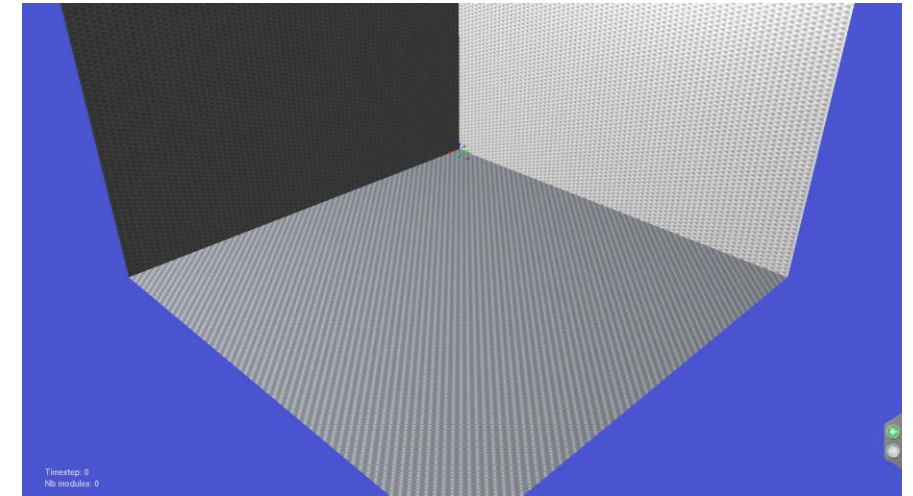
Comparison with a centralized system

- Exemple of Dijkstra algorithm
 - Only one computer
 - All vertices are reachable from the computer

```
function Dijkstra(Graph, source):  
  
  create vertex set Q  
  
  for each vertex v in Graph:  
    dist[v] ← INFINITY  
    prev[v] ← UNDEFINED  
    add v to Q  
  dist[source] ← 0  
  
  while Q is not empty:  
    u ← vertex in Q with min dist[u]  
    remove u from Q  
    for each neighbor v of u still in Q:  
      alt ← dist[u] + length(u, v)  
      if alt < dist[v]:  
        dist[v] ← alt  
        prev[v] ← u  
  
  return dist[], prev[]
```

Basic of distributed programs

- All program mainly use the basic treatments:
 - Flooding information from a leader to all the other modules
 - Receiving a message from a subset of modules
 - Sending a message following a pre-defined path in the graph.



Flooding an information into the graph

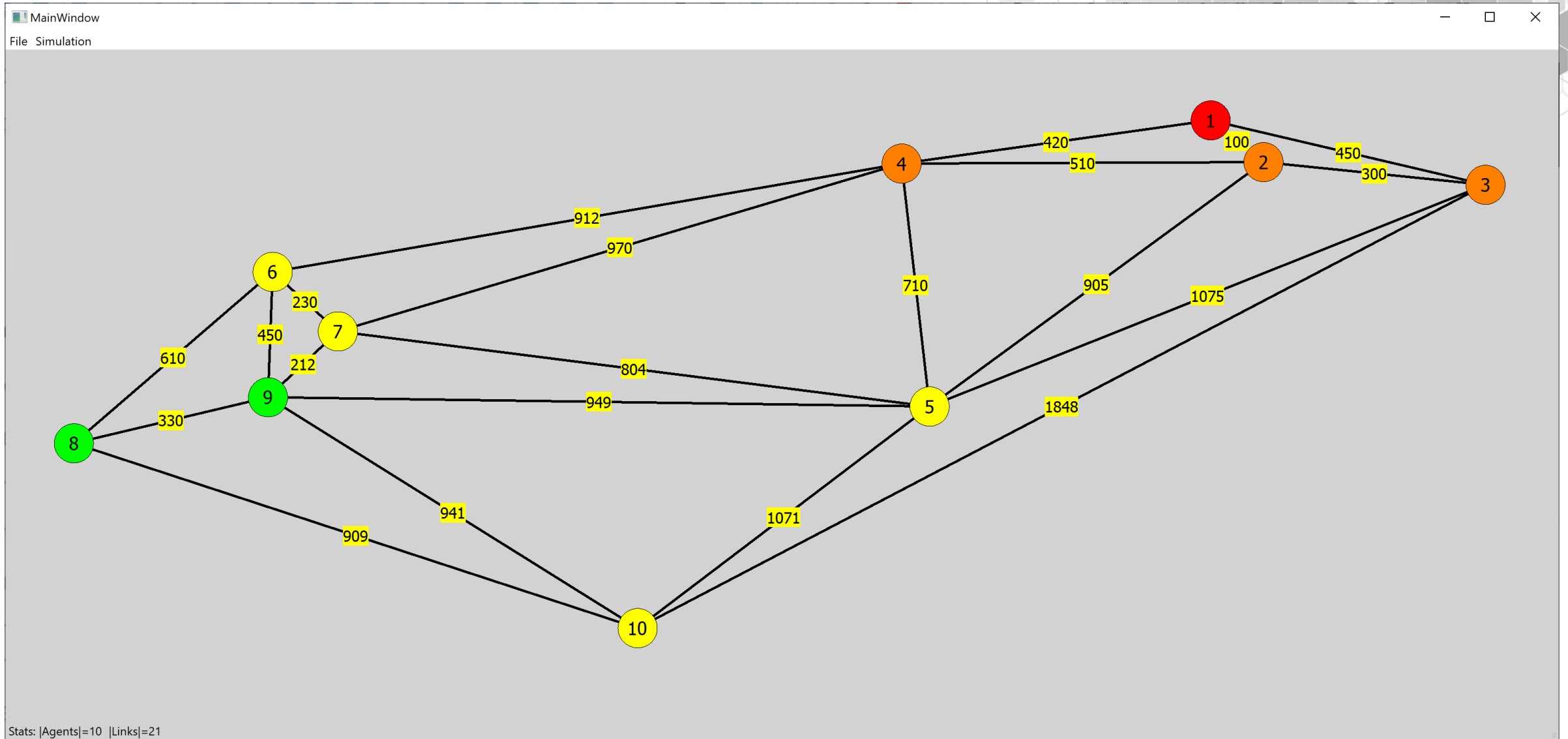
- We consider a leader that will send a message to all the modules to the other agents
- Create a gradient of values from the leader
 - Local memory: distance variable
- Algorithm:
 - The leader starts with distance=0, other with distance= ∞
 - The leader sends a message `DISTANCE(1)` to all its neighbors
 - When a module receives a `DISTANCE(d)` message from a sender
 - If ($d < \text{distance}$) {
 - distance=d;
 - Send a message `DISTANCE(d+1)` to all its neighbors but the sender

Coding this program

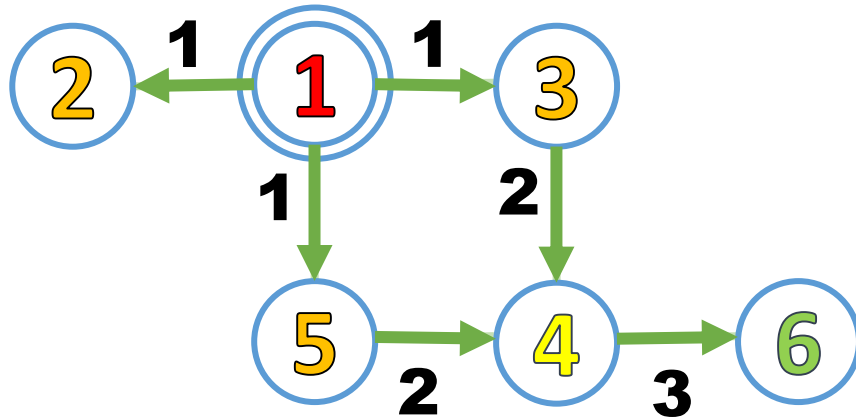
```
void MyAgent::start() {
    if (getId()==1) {
        setColor(0);
        myDistance=0;
        sendMessageToAllNeighbors(new MessageOf<int>(DISTANCE_MSG, 1) );
    } else {
        setColor(9);
        myDistance=std::numeric_limits<unsigned int>::max();
    }
}

void MyAgent::messageCallback(MessagePtr ptr, unsigned int senderId) {
    MessageOf<int>*msg = (MessageOf<int>*)ptr.get();
    unsigned int distMsg = msg->getData();
    qDebug() << getId() << " rec: " << distMsg;
    if (distMsg<myDistance) {
        myDistance=distMsg;
        setColor(myDistance);
        sendMessageToAllNeighbors(new MessageOf<int>(DISTANCE_MSG, distMsg+1), senderId);
    }
}
```

Demo

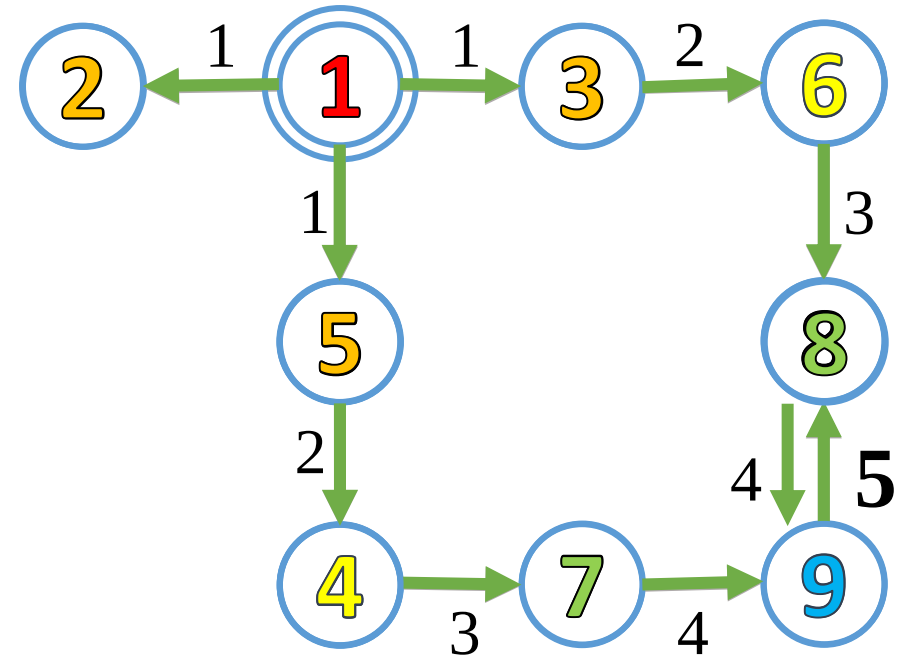
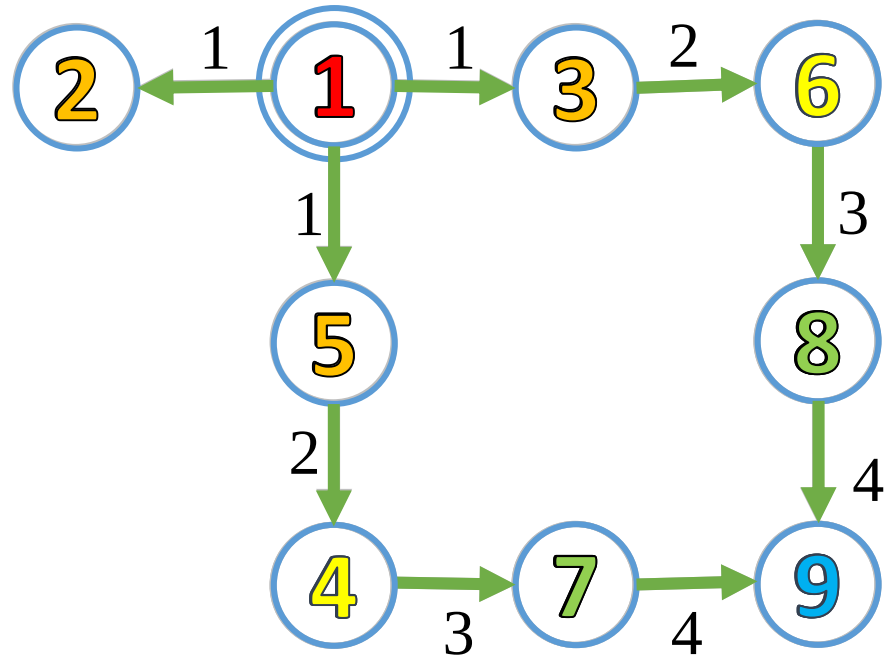


Simple Flooding algorithm



Limit of the algorithm:
The leader is never notified that the flooding is completed.

Order of messages received



How to send an acknowledgment message

- Each Agent stores the list of neighbors to which it has sent a distance
 - The initial leader sends its messages (clear `myWaitingFrom`)
 - When a « Distance message » arrives with `Message.distance < myDistance`
 - Store the sender Id in `myParent`
 - If it is not the first time send a « Back Message » to the previous parent
- If `distMesg >= myDistance`
 - Send a « Back Message » to the sender
- When a « Back Message » arrives
 - Remove the sender Id from the `myWaitingFrom` list
- When the `myWaitingFrom` list is empty
 - Send a « Back Message » to the `myParent` if it has been set
 - If it does not been set the algorithm has reached the leader.

```

const int DISTANCE_MSG=1;
const int BACK_MSG=2;
const unsigned int NO_PARENT=0;
MyAgent::MyAgent(unsigned int p_id,const QPointF &p_position):Agent(p_id,p_position) {}
void MyAgent::start() {
    qDebug() << "start " << getId();
    myParent=NO_PARENT;
    myWaitingFrom.clear();
    setBlink();
    if (getId()==1) {
        setColor(0); myDistance=0;
        // send message to all neighbors and store the list
        sendMessageToAllNeighbors(new MessageOf<int>(DISTANCE_MSG,1),myWaitingFrom);
    } else {
        setColor(9); myDistance=std::numeric_limits<unsigned int>::max();
    }
}

void MyAgent::messageCallback(MessagePtr ptr, unsigned int senderId) {
    switch (ptr->getId()) {
        case DISTANCE_MSG: distanceMsgCallback(ptr,senderId); break;
        case BACK_MSG: backMsgCallback(ptr,senderId); break;
    }
}

```

Distance message callback

```
void MyAgent::distanceMsgCallback(MessagePtr ptr, unsigned int senderId) {
    MessageOf<int>*msg = (MessageOf<int>*)ptr.get();
    unsigned int distMsg = msg->getData();
    if (distMsg<myDistance) {
        myDistance=distMsg;
        setText(QString::number(getId()) + ":" +QString::number(myDistance));
        setColor(myDistance);
        if (myParent!=NO_PARENT) {
            sendMessageTo(myParent,new Message(BACK_MSG));
        }
        myParent=senderId;
        sendMessageToAllNeighbors(new MessageOf<int>(DISTANCE_MSG,distMsg+1),myWaitingFrom,senderId);
        if (myWaitingFrom.empty()) {
            sendMessageTo(myParent,new Message(BACK_MSG));
        }
    } else {
        sendMessageTo(senderId,new Message(BACK_MSG));
    }
}
```


Back message callback

```
void MyAgent::backMsgCallback(MessagePtr ptr, unsigned int senderId) {  
    myWaitingFrom.removeOne(senderId);  
    if (myWaitingFrom.empty()) {  
        if (myParent!=NO_PARENT) {  
            sendMessageTo(myParent, new Message(BACK_MSG));  
        } else {  
            setBlink(0,10);  
        }  
    }  
}
```

Spanning tree



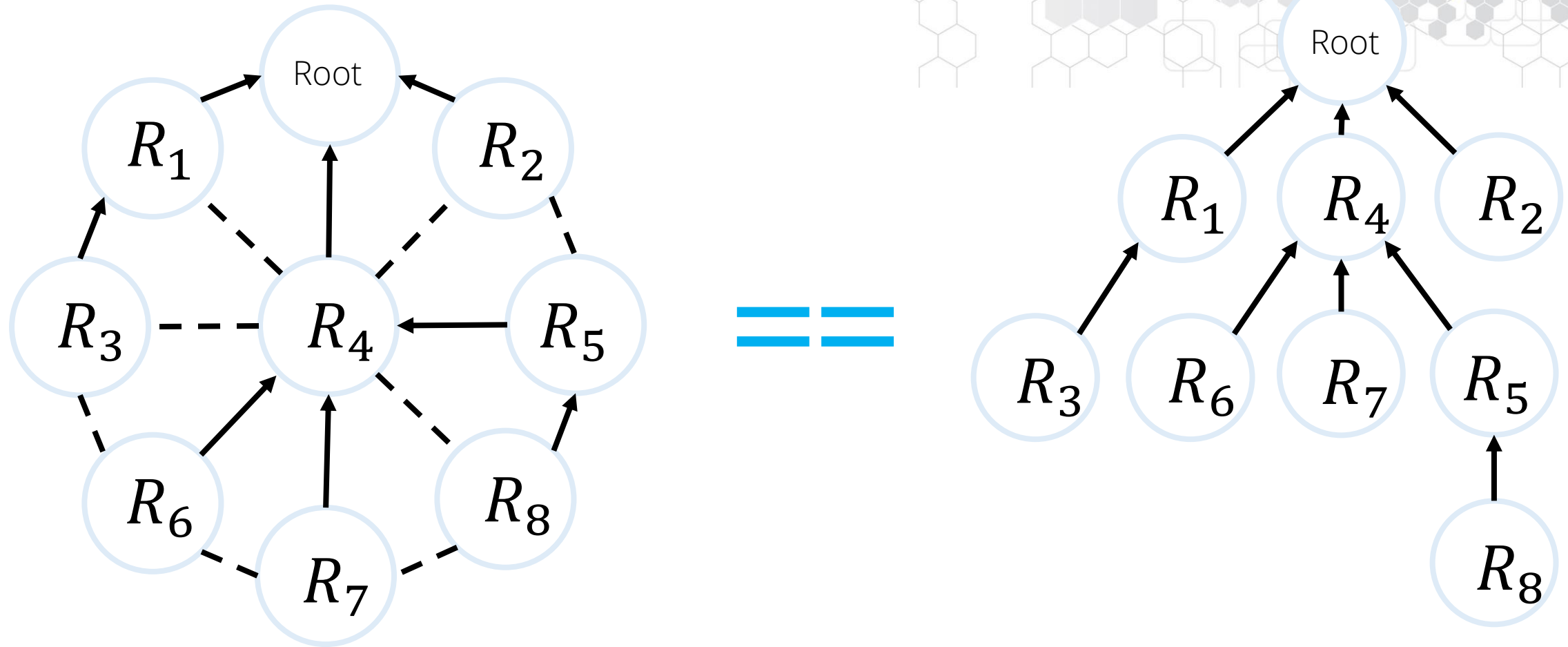
- A spanning tree is a tree covering the graph of agents
- The root is placed at the leader
- It allows to remove loops to send message from the leader to all the other modules

Minimal spanning tree



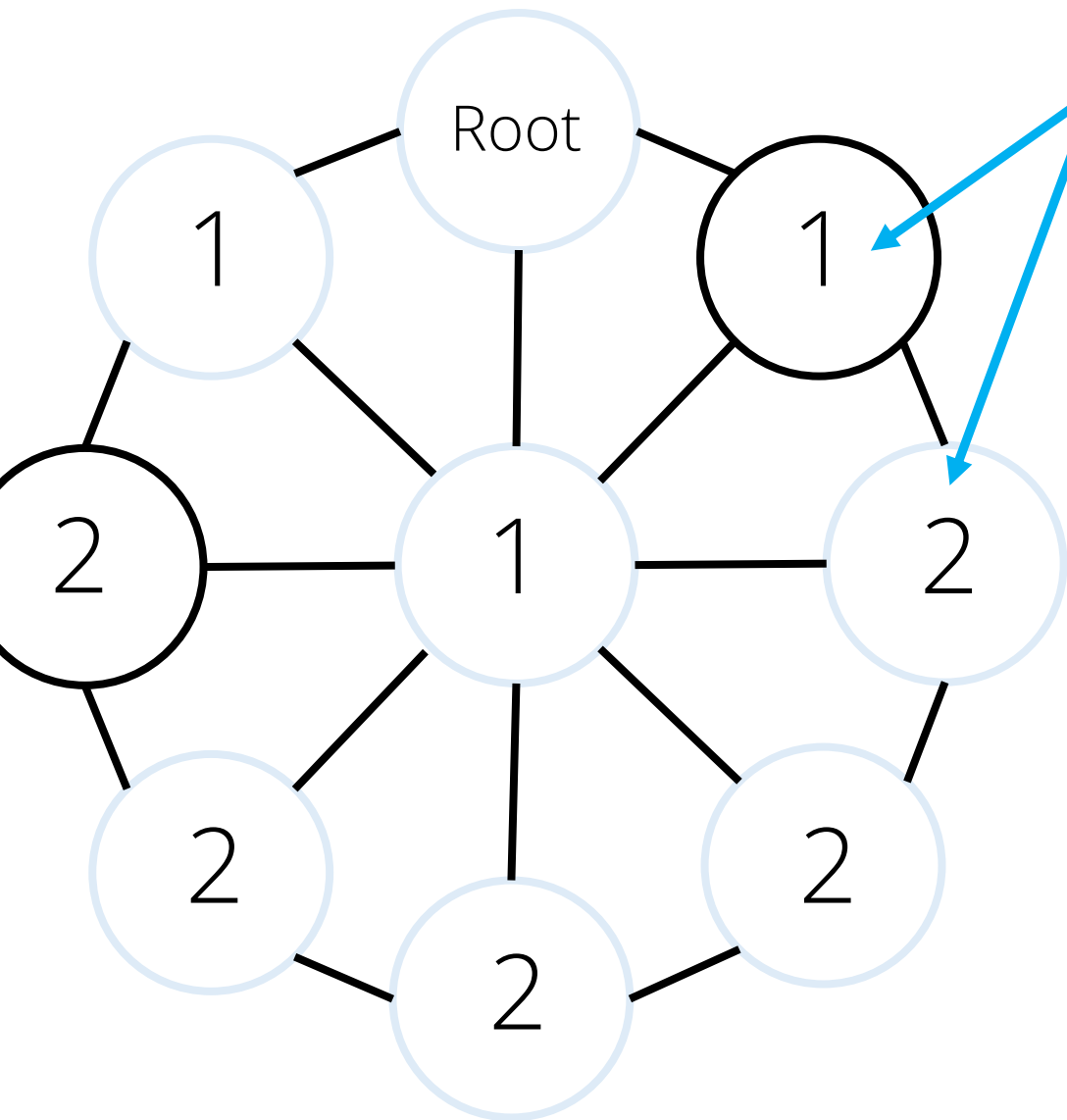
- A minimum spanning tree (MST) or minimum weight spanning tree is a subset of the edges of a connected, edge-weighted undirected graph that connects all the vertices together, without any cycles and with the minimum possible total edge weight.
- That is, it is a spanning tree whose sum of edge weights is as small as possible.

Rooted spanning tree construction





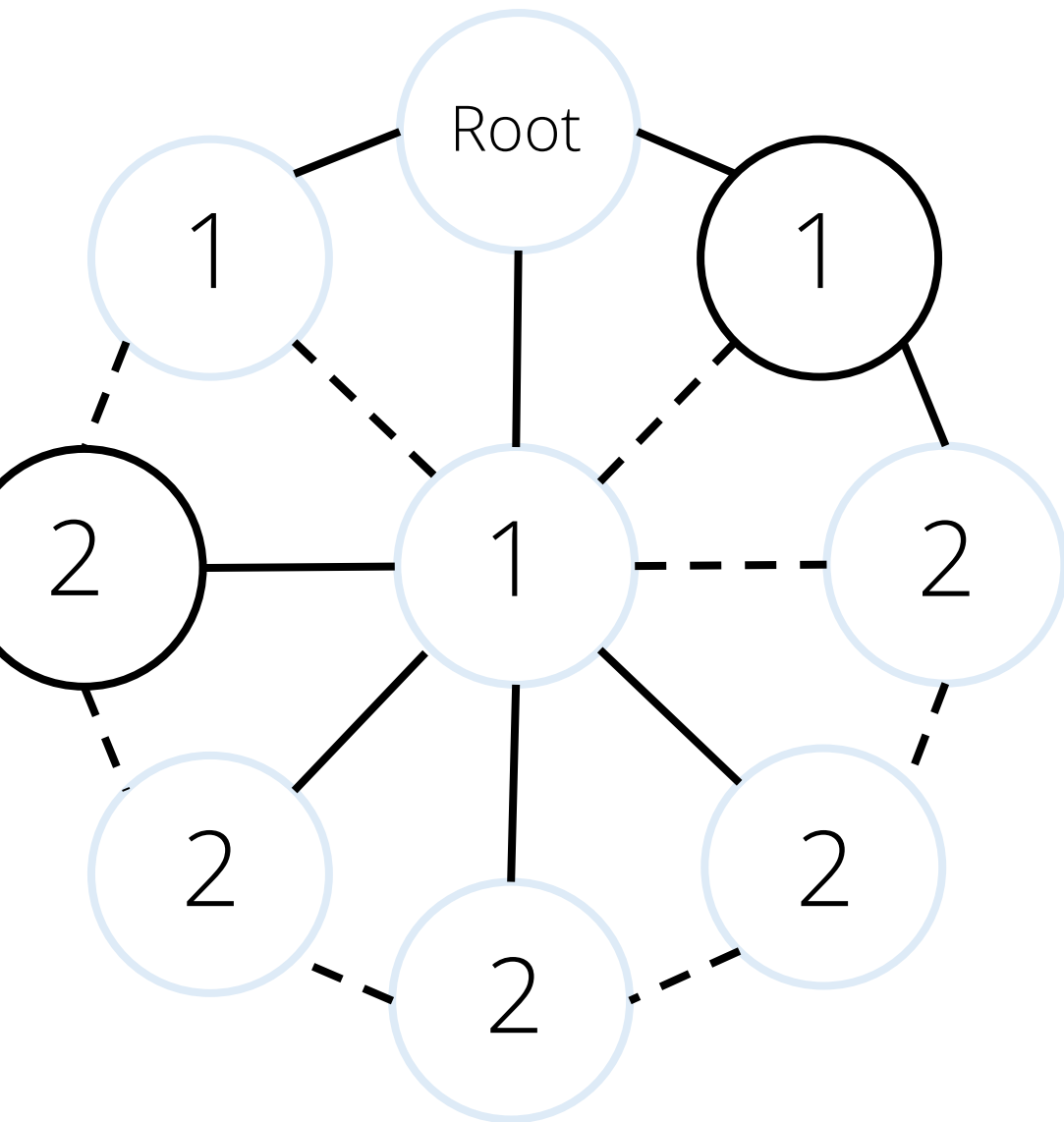
Distributed Breadth-First Spanning Tree



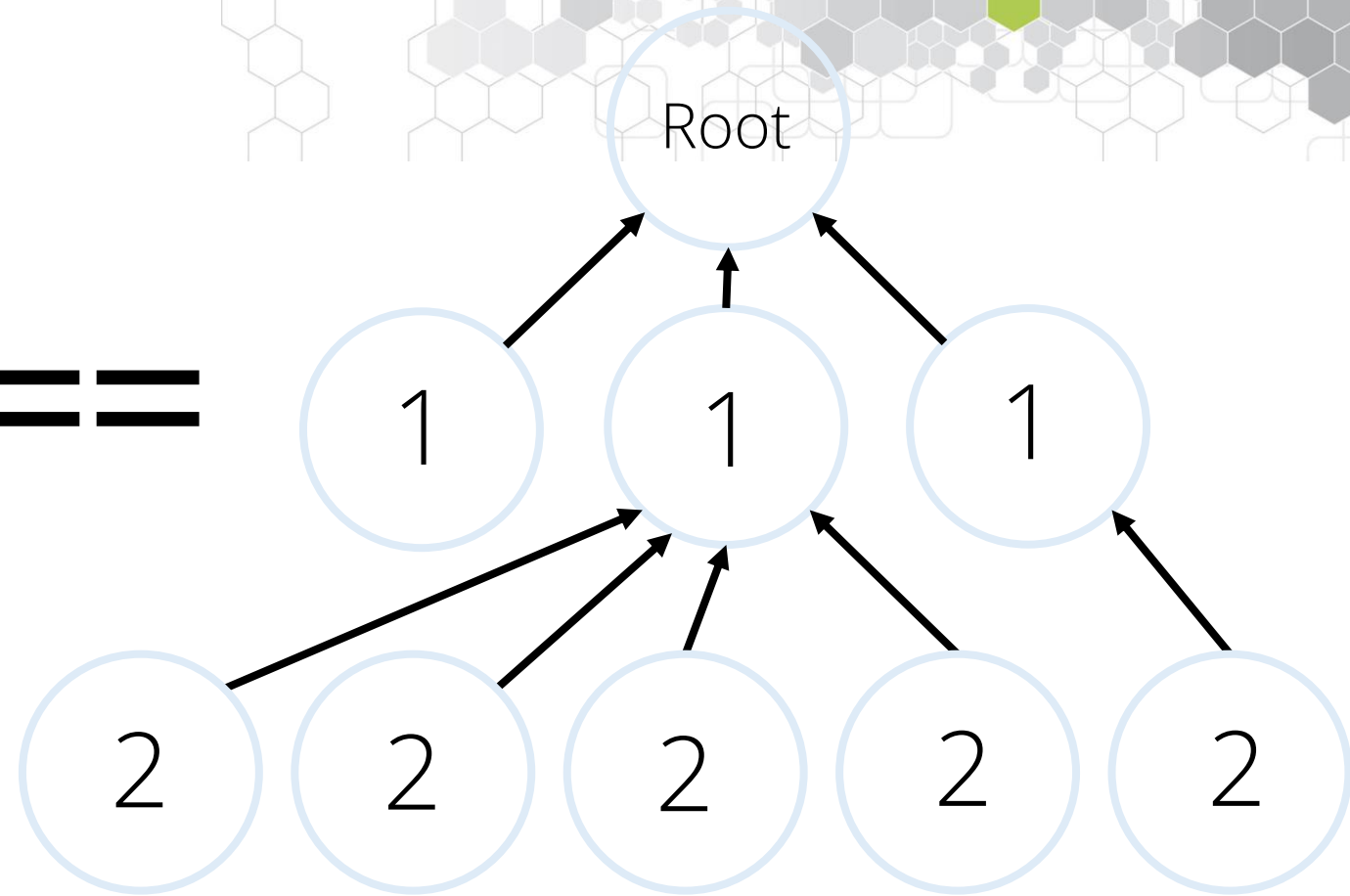
Min distance to the root



Distributed Breadth-First Spanning Tree



==



Breadth-First Spanning Tree algorithm with centralized control

```
when START() is received do
  parenti ← i;
  childreni ← ∅;
  distancei ← 0;
  to_sendi ← neighborsi;
  for each k ∈ to_sendi do
    send GO(0) to pk
  waiting_from i ← neighborsi
```

Michel Raynal

Distributed Algorithms for Message-Passing Systems

Springer

Michel Raynal, *Distributed Algorithms for Message-Passing Systems*, Springer, 2013.

Breadth-First Spanning Tree algorithm with centralized control

```
when GO(d) is received from  $p_j$  do
    if ( $\text{parent}_i = \perp$ ) then
         $\text{parent}_i \leftarrow j$ ;  $\text{children}_i \leftarrow \emptyset$ ;  $\text{distance}_i \leftarrow d+1$ ;
         $\text{to\_send}_i \leftarrow \text{neighbors}_i \setminus \{j\}$ ;
        if ( $\text{to\_send}_i = \emptyset$ ) then
            send BACK(stop) to  $p_j$ 
        else
            send BACK(continue) to  $p_j$ 
    else
        if ( $\text{parent}_i = j$ ) then
            for each  $k \in \text{to\_send}_i$  do
                send GO( $\text{distance}_i$ ) to  $p_k$ 
                 $\text{waiting\_from}_i \leftarrow \text{to\_send}_i$ 
            else
                send BACK(no) to  $p_j$ 
```

First time this node is explored

End of the wave and no further expansion needed

End of the wave and further expansions needed

Breadth-First Spanning Tree algorithm with centralized control

```
when BACK(resp) is received from  $p_j$  do
    waiting_fromi ← waiting_fromi \ { j } ;
    if resp ∈ {continue, stop} then childreni ← childreni ∪ { j }
    if resp ∈ {stop, no} then to_sendi ← to_sendi \ { j }
    if (to_sendi = ∅) then ← End of the algorithm
        if (parenti = i) then ← Tree cannot be expanded from  $p_i$ 
            // the root learns that the tree is built
        else send BACK(stop) to  $p_{parent_i}$ 
    else
        if (waiting_fromi = ∅) then
            if (parenti = i) then ← Starting a new wave
                for each  $k \in to\_send_i$  do
                    send GO(distancei) to  $p_k$ 
                    waiting_fromi ← to_sendi
            else send BACK(continue) to  $p_{parent_i}$ 
```

Implement a simulator of distributed system

- Agent code
 - Class Agent
 - id
 - Private local variable
 - List of P2PLinks
 - Start method
 - MessageCallBack function
 - setColor, setText
 - sendMessageToAllNeighbors
 - sendMessageTo
- World code
 - Class P2PLink
 - Duration
 - Agent A1,A2
 - sendMessage
 - Class Message<type>
 - TypeOfMsg, data, senderId
 - Class World
 - Store the graph
 - List of timed message

Elect an agent at the center of the graph

- Here the center is a vertex which is the best placed to send a message to all the others.
- First choose an initial leader (A)
 - From its id.
 - Using the tap event.
- Find the module furthest from A (B)
- Search the module furthest from B (C)
- Select the module X for which $|d(X, B) - d(X, C)| \leq 1$
 - Where $d(X, B)$ is the distance between X and B

Leader Election



- Create a graph $G(V,E)$
 - Each vertex V
 - Unique ID
 - Each edge E
 - Delay of communication
- Elect a leader
 - The leader is the module with the highest ID
 - After the algorithm
 - All modules know that the election is over.
 - All modules know the link to follow to reach the leader.
 - The leader knows he is the leader.
 - Propose a method that gives a solution if it can exist one duplicated ID.